Requirements Engineering

Lecture 3a

Agenda

- Basic Concepts
- Problems in Requirements Engineering
- Communication Principles
- Functional and Non-Functional Requirements

Basic Concepts

What is Requirements Engineering?

- The requirements for a system are the descriptions of the services that a system should provide and the constraints on its operation.
- These requirements reflect the needs of customers for a system that serves a certain purpose such as controlling a device, placing an order, or finding information.
- The process of finding out, analysing, documenting and checking these services and constraints is called requirements engineering.

User and System Requirements

- According to Sommerville, user requirements mean the high-level abstract requirements (problem domain) and system requirements mean the detailed description of what the system should do (solution domain).
- User requirements are statements, in a natural language plus diagrams, of what services the system is expected to provide to system users and the constraints under which it must operate.
- System requirements are more detailed descriptions of the software system's functions, services, and operational constraints. The system requirements document (sometimes called a functional specification) should define exactly what is to be implemented.

User and System Requirements

- A user requirement may be expanded into several system requirements.

User requirements definition

1. The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

1.1 On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
1.2 The system shall generate the report for printing after 17.30 on the last working day of the month.
1.3 A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
1.4 If drugs are available in different dose units (e.g. 10mg, 20mg, etc.) separate reports shall be created for each dose unit.
1.5 Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

Problems in Requirements Engineering

Problems in Requirements Engineering

- Stakeholders don't know what they want and/or they cannot articulate their requirements in language the requirements engineer can understand
  - It is difficult to visualise the end-product, particularly if innovative
- Stakeholders fail to define some requirements, because they believe the requirement to be obvious
- Stakeholders think they know what they want until we give them what they said they wanted
- Analysts think they understand stakeholder problems better than the stakeholder

Problems in Requirements Engineering

- We never know whether we have defined 'all' the requirements
- Different stakeholders have different viewpoints and hence different requirements, which may conflict.
- The business environment is dynamic, and requirements may change
- The requirements engineer & stakeholders can easily fall into the trap of visualising solutions rather than defining what needs to be done

Problems in Requirements Engineering

- The requirements engineer may not truly understand the business problem & propose an inadequate solution or accept the customer solution.
- Most software engineering products are derived from previous models, but requirements have to be created.
- There are often no physical constraints to limit the imagination

Communication Principles

Communication Principles

- Before customer requirements can be analysed, modelled, or specified they must be gathered through the communication activity.
- Principle 1. Listen.
  - Ask for clarification if something is unclear but avoid constant interruptions.
- Principle 2. Prepare before you communicate.
  - Spend the time to understand the problem before you meet with others.
- Principle 3. Someone should facilitate the activity.
  - To keep the conversation moving in a productive direction; to mediate any conflict that does occur; and to ensure that other principles are followed

Communication Principles

- Principle 4. Face-to-face communication is best.
  - It usually works better when some other representation of the relevant information is present.
- Principle 5. Take notes and document decisions.
  - Someone participating in the communication should serve as a “recorder” and write down all important points and decisions
- Principle 6. Strive for collaboration.
  - Each small collaboration serves to build trust among team members and creates a common goal for the team

Communication Principles

- Principle 7. Stay focused; modularize your discussion.
  - The more people are involved in any communication, the more likely that discussion will bounce.
- Principle 8. If something is unclear, draw a picture.
  - A sketch or drawing can often provide clarity when words fail to do the job.
- Principle 9. (a) Once you agree to something, move on. (b) If you can't agree to something, move on. (c) If a feature or function is unclear and cannot be clarified at the moment, move on.
- Principle 10. Negotiation is not a contest or a game. It works best when both parties win.

Functional and Non-Functional Requirements

Functional and Non-Functional Requirements

- Software system requirements are often classified as functional or non-functional requirements.
- Functional requirements
  - These are statements of services the system should provide, how the system should react to particular inputs, and how the system should behave in particular situations.
  - In some cases, the functional requirements may also explicitly state what the system should not do.
  - Functional requirements should be expressed quantitatively whenever possible to avoid conflict with the client.

Functional Requirements

- Examples of functional requirements for a system, used to maintain information about patients receiving treatment for mental health problems:
  - A user shall be able to search the appointments lists for all clinics.
  - The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
  - Each staff member using the system shall be uniquely identified by his or her eight-digit employee number.

Functional Requirements

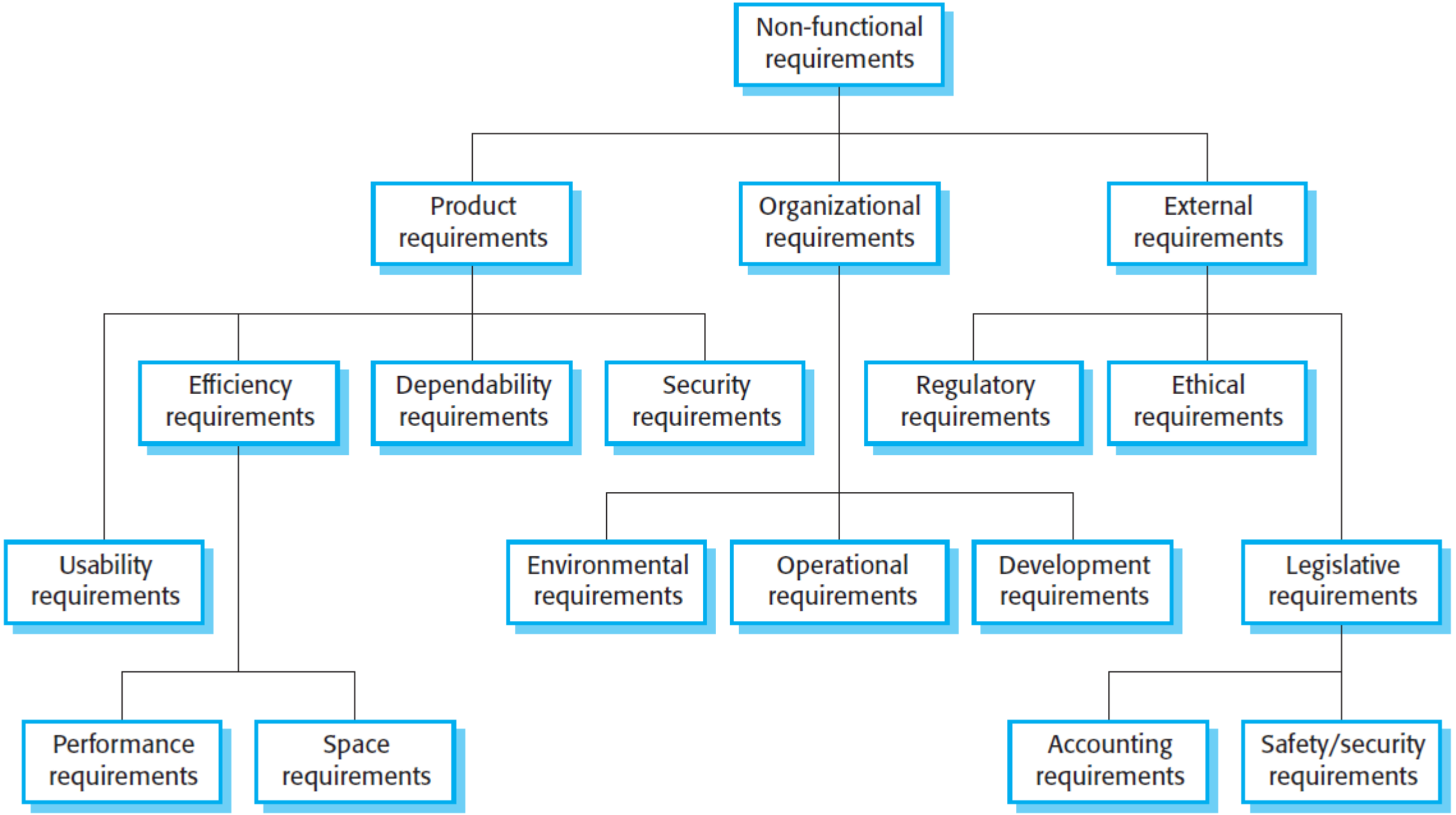
- More examples:
  - Cuando el usuario seleccione la opción “modificar detalles del bote”, el sistema solicitará el ingreso del nombre del bote.
  - El sistema debe mostrar un ícono en el mapa que presente la ubicación actual del vehículo sobre el mapa, y además debe visualizarse el recorrido del mismo durante la última hora.
  - Al dar click sobre el ícono del vehículo en el mapa debe mostrarse una ventana que muestre su placa, velocidad, rumbo, latitud, longitud, y hora de envío de su última ubicación.

Exercise

- Encuentre ocho *ambigüedades*, *conflictos* u *omisiones* en el siguiente párrafo de especificación de requerimientos de una parte de un sistema bibliotecario basado en software para que estudiantes puedan extender el periodo de préstamos de libros.
  - *Un estudiante podrá extender el periodo de préstamo de libros en préstamo en cualquier momento. El sistema puede aprobar o rechazar una extensión y debe notificar al prestamista de su decisión. El sistema debe estar disponible a toda hora.*
- Usando lenguaje natural, defina seis requerimientos de usuario para el sistema descrito, los mismos que deben mejorar la claridad en la descripción de lo requerido. Usted puede hacer cualquier asunción razonable acerca de los requerimientos.

Non-Functional Requirements

- Non-functional requirements
  - These are constraints on the services or functions offered by the system. They include timing constraints, constraints on the development process, and constraints imposed by standards. Non-functional requirements often apply to the system as a whole rather than individual system features or services.


Non-functional requirements
Product requirements
Organizational requirements
External requirements
Efficiency requirements
Dependability requirements
Security requirements
Regulatory requirements
Ethical requirements
Usability requirements
Environmental requirements
Operational requirements
Development requirements
Legislative requirements
Performance requirements
Space requirements
Accounting requirements
Safety/security requirements

Non-Functional Requirements

- Product requirements
  - Efficiency: performance, space
  - Usability: easy of use, error tolerance
  - Dependability: availability, reliability, robustness, repairability, maintainability
  - Security: resistance to deliberate or accidental intrusion, survivability under attack

Non-Functional Requirements

- Product (Efficiency)
  - Efficiency requirements relate to performance and space and answer such questions as, 'what performance criteria are required by this system', 'are there capacity restrictions', ‘will projected growth affect performance or capacity’?
  - Measurable requirements to consider include:
    - ❖Average response time for a transaction
    - ❖Maximum response time for a transaction
    - ❖Throughput: transactions per second
    - ❖Capacity: the number of transactions the system should cope with
    - ❖Screen refresh time
    - ❖Space: Kbytes
    - ❖Number of RAM chips

Non-Functional Requirements

- Product (Usability)
  - Usability requirements are concerned with specifying the user interface and end-user interactions with the system.
  - Measurable requirements to consider include:
    - ❖Hours of training required before independent use of the system (defined by category of user);
    - ❖Measurable task times for typical tasks (for a normal user);
    - ❖Conforms to the standards defined in the XYZ ‘look & feel’ standard
    - ❖Existence of user manuals, online help etc;
    - ❖Errors made when working at a normal rate;
    - ❖The system should be considered by x% of the user community to be as usable as ? system;
    - ❖Times to install/uninstall software;
    - ❖Times to correct an error (in response to an error message - how helpful is the message?);
    - ❖Time to contact and receive a helpful response from the service provider.

Non-Functional Requirements

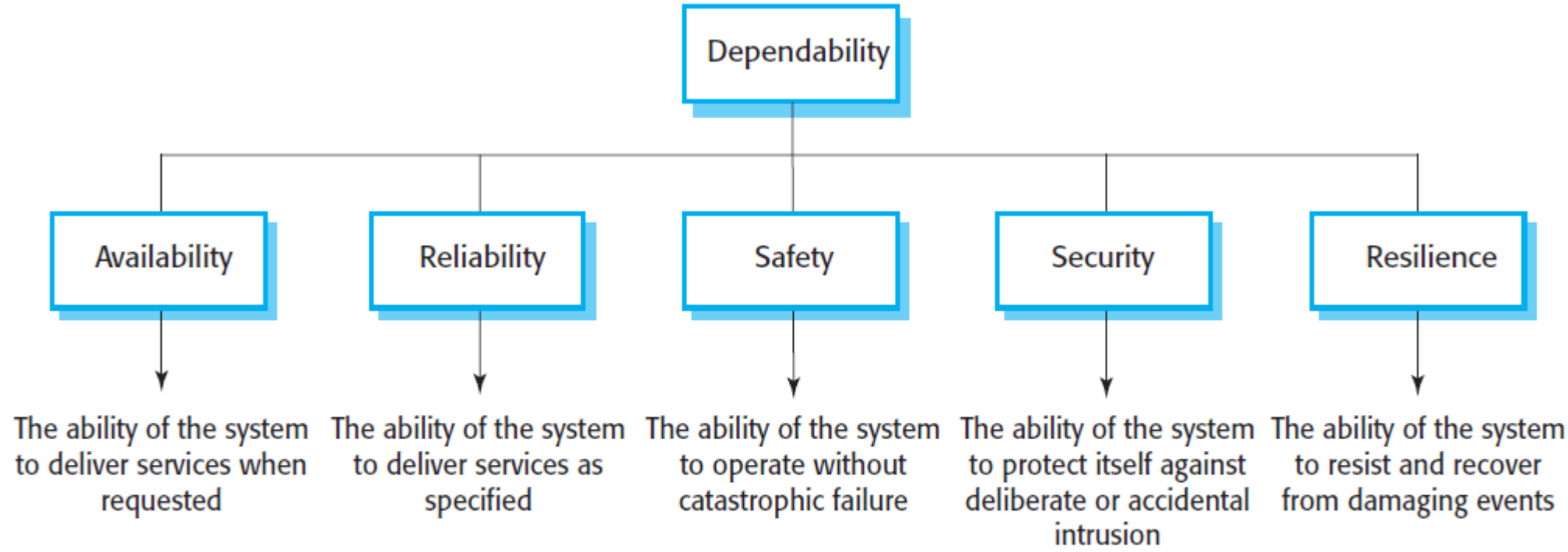


Figure 10.1 Principal dependability properties

Non-Functional Requirements

Other system properties are closely related to these five dependability properties and influence a system's dependability:

1. *Repairability* System failures are inevitable, but the disruption caused by failure can be minimized if the system can be repaired quickly. It must be possible to diagnose the problem, access the component that has failed, and make changes to fix that component. Repairability in software is enhanced when the organization using the system has access to the source code and has the skills to make changes to it. Open-source software makes this easier, but the reuse of components can make it more difficult.

2. *Maintainability* As systems are used, new requirements emerge, and it is important to maintain the value of a system by changing it to include these new requirements. Maintainable software is software that can be adapted economically to cope with new requirements, and where there is a low probability that making changes will introduce new errors into the system.

3. *Error tolerance* This property can be considered as part of usability and reflects the extent to which the system has been designed, so that user input errors are avoided and tolerated. When user errors occur, the system should, as far as possible, detect these errors and either fix them automatically or request the user to re-input their data.

Non-Functional Requirements

- Product (dependability) - Maintainability/Repairability
  - Maintainability requirements are concerned with the ability to easily modify the software and carry out repairs and links to the non-functional requirement type repairability.
  - Factors that affect maintainability include response time to carry out change (mean time to change) and category of potential change.
  - A measurable requirement would specify response time to carry out a change of type: simple, moderate, complex.

Non-Functional Requirements

- Product (dependability) - Reliability/availability/robustness
  - Reliability requirements are concerned with identifying the potential types of system failure which, may occur and defining the degree of reliability demanded by the user for each of these circumstances.
  - In order to achieve the specified reliability, functional requirements may need to be specified to reduce the potential for the types of system failure to occur.
  - Measurable requirements include:
    - ❖Mean time to failure (MTTF in units of time for each type of failure)
    - ❖Probability of unavailability
    - ❖Rate of occurrence of failure
    - ❖Availability (%availability between ?am and ? pm for all/% of users)
    - ❖Time to restart after failure (MTTR in units of time for each type of failure - % of users for whom the system is available may also be specified)
    - ❖Percentage of events causing failure
    - ❖Probability of data corruption on failure
    - ❖Defect rate (defects/FP), defects per type: critical, significant, minor, etc.

Non-Functional Requirements

- Product (Security)
  - Security requirements are usually identified by analysing the threats to the system and identifying potential solutions. Additional functional requirements may be generated.
  - In terms of security, design constraints may be specified e.g., encryption of sensitive data.
  - The analyst asks questions like 'how secure does the data need to be'?

Non-Functional Requirements

- Organisational requirements
  - Environmental: operating platform, portability
    - ❖Number of target platforms
    - ❖Percentage of platform dependent statements
    - ❖Time to adapt the software to a new platform for simple, medium and complex systems.
  - Operational: delivery time & cost, how used, interoperability
    - ❖Operational requirements may relate to the environment in which the product operates, for example, might define the range of operating temperatures for the equipment.
    - ❖Delivery requirements specify when the system and all associated deliverables must be available to the stakeholders.
    - ❖Interoperability defines how the system interacts with external systems. This may include specification of hardware and software interfaces.
  - Development: language, design method, process standards
    - ❖The requirements should specify any standards which must be used, for example, communication standards (TCP/IP), platform standards (Windows, UNIX), quality standards (ISO 9001, CMM), project management (PRINCE2).
    - ❖The requirements could also define internal standards e.g. use of a particular CASE tool, object-oriented development, particular programming languages or design methods.

Non-Functional Requirements

- External requirements
  - Regulatory
    - ❖Identify any regulatory bodies which will impact on the requirements, for example the Financial Services Authority for accounting practices, Advertising Standards Authority which may affect web services.
  - Legislative: accounting, safety, security
    - ❖Within legislative requirements we need to consider whether there are any legal requirements (e.g., accounting rules), privacy (security) or health & safety issues which will impact requirements.
  - Ethical: acceptable to customer and public
    - ❖For example, if there are privacy issues relating to information handled by the system, the steps to guard privacy of the individual must be specified. For example, in insurance systems is it ethical to record private information linked with legal risk e.g., medical history?

Non-Functional Requirements

- Los requerimientos no funcionales deben ser escritos en lenguaje natural.
- Además, deben ser expresados **cuantitativamente**. Es decir, se debe utilizar métricas.
  - De esta manera, pueden ser ***validados por el cliente*** para evitar conflictos.

Non-Functional Requirements

- El operador debe ser capaz de usar el sistema con *media hora* de entrenamiento:
  - Product (usability)
- Las respuestas a requerimientos de estimación deben tomar menos de *2 segundos*:
  - Product (performance)

Non-Functional Requirements

- El sistema debe estar disponible el 99% del tiempo entre las 08h00 y la media noche.
  - Product (dependability) – availability/reliability
- Las mejoras al sistema deben ser completadas en una semana
  - Product (dependability) - maintainability

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Non-Functional Requirements

- El tiempo promedio de respuesta debe ser menor a 5 segundos
  - Product (efficiency) - performance
- El nuevo sistema debe estar disponible para el inicio del proyecto XYZ
  - Organizational (operational) – delivery

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Non-Functional Requirements

- Un director de proyecto experimentado debe ser capaz de usar el nuevo sistema luego de una hora de entrenamiento
  - Product (usability) – easy of use
- Use la librería de clases en el servidor de desarrollo
  - Organisational (development)

Non-Functional Requirements

- El sistema debe operar sobre las plataformas Windows 11/10 y Linux
  - Organizational (environmental) – operating platform/portability
- El sistema debe ser accesible únicamente para usuarios autorizados
  - Product (security)/External (ethical) – genera requerimientos funcionales

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Non-Functional Requirements

- Las estimaciones y hechos serán utilizados para actualizar la base de datos histórica de datos de estimaciones mantenidos por el sistema CostXpert
  - Organisational (operational) – interoperability
- El sistema debe cumplir los requerimientos en la ley de protección de datos
  - External (Legislative)-security

© Dra. Villavicencio, © Dr. Mera-Gómez
2026

Non-Functional Requirements

- En ocasiones los requerimientos no funcionales derivan nuevos requerimientos funcionales. Por ejemplo:
- El sistema debe ser accedido únicamente por usuarios autorizados. Product (security)
  - Se requiere una ventana que permita actualizar la lista de usuarios autorizados
  - Rechazar el acceso de usuarios no autorizados con el mensaje XYZ
  - Comparar los detalles de login contra la lista de usuarios autorizados.

Metrics for Non-Functional Requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Exercise

- Considere el siguiente párrafo de especificación de requerimientos de una parte de un sistema bibliotecario basado en software para que estudiantes puedan extender el periodo de préstamos de libros.
  - *Un estudiante podrá extender el periodo de préstamo de libros en préstamo en cualquier momento. El sistema puede aprobar o rechazar una extensión y debe notificar al prestamista de su decisión. El sistema debe estar disponible a toda hora*.
- Defina dos requerimientos no funcionales que podrían ser identificados para el sistema descrito y explique las razones por la cuales usted considera estos requerimientos apropiados para el sistema. Para cada requerimiento especifique el tipo de requerimiento no funcional utilizando la categorización de Sommerville para requerimientos no funcionales.

Next On

Take Away Points

- What is requirements engineering?
- What is the difference between functional and non-functional requirements?
- Provide examples for each kind of non-funcional requirement in the classification of Sommerville.
- Specify requirements in such a way they are unambiguous, complete and consistent.

Further Reading

- Pressman and Maxim, “Software Engineering”
  - Chapter 7: Principles that Guide Practice
  - Chapter 8: Understanding Requirements
- Ian Somerville, “Software Engineering”
  - Chapter 4: Requirements Engineering
  - Chapter 10: Dependable Systems
- S. Robertson and J. Robertson, “Mastering the Requirements Process (third edition)”
- Mike Cohn, “User Stories Applied for Agile Software Development”

Further Reading

- K. Wiegers and J. Beatty, “Software Requirements (Developer Best Practices) 3rd edition”
- Dean Leffingwell, “Agile Software Requirements: Lean requirements practices for teams, programs, and the enterprise”
- Acknowledgement: Some content in these slides is a courtesy of Joan Jackson

Next Lecture

- Techniques Used In Requirements Elicitation and Analysis
- Requirements and User Stories